A FRIENDLY GUIDE

Individual Guide

Start here, learn by doing, and find the task you need

Beginner edition for release 0.4.0 · 19 October 2026

58 task types · 22 real projects · No R experience needed

Contents

1 Welcome to Individual Guide
- 1.1 Find your starting point
- 1.2 Six words you will see often
- 1.3 How to follow this guide

2 Installation and first launch
- 2.1 Download the public Windows release
- 2.2 Open the app each day
- 2.3 Update the app
- 2.4 Before your first project

3 Your first project
- 3.1 Open the flower example
- 3.2 Look around before running anything
- 3.3 Make one small change
- 3.4 Find the report and log
- 3.5 Build a small flow yourself

4 Find your way around
- 4.1 Open a task or result
- 4.2 Understand the arrows
- 4.3 Choose useful names

5 Save, reopen, and share projects
- 5.1 Save your project
- 5.2 Move work to another computer
- 5.3 Recover interrupted work
- 5.4 Share a project with care

6 Run tasks and check results
- 6.1 Run, Preview, and Save do different jobs
- 6.2 Check the task status
- 6.3 Run the right part of a project
- 6.4 Stop a run

7 Importing your data
- 7.1 Import Data
- 7.2 Work with a database

8 Select, join, and calculate data
- 8.1 Query Builder
- 8.2 Filter and Sort
- 8.3 Derive Columns

9 Optional: saved code steps
- 9.1 Script

10 Clean and reshape your data
- 10.1 Append Table
- 10.2 Compare Data
- 10.3 Stack Columns
- 10.4 Split Columns
- 10.5 Transpose
- 10.6 Rank
- 10.7 Standardize Data
- 10.8 Random Sample
- 10.9 Remove Duplicates
- 10.10 Keep / Drop Columns
- 10.11 Rename Columns
- 10.12 First / Last N Rows
- 10.13 Replace Missing Values
- 10.14 Recode Values
- 10.15 Change Column Types

11 Get to know your data
- 11.1 Characterize Data
- 11.2 List Data
- 11.3 One-Way Frequencies
- 11.4 Summary Statistics
- 11.5 Summary Tables
- 11.6 Table Analysis
- 11.7 Distribution Analysis

12 Make charts people can read
- 12.1 Keep the output you need
- 12.2 Bar Chart
- 12.3 Bar-Line Chart
- 12.4 Box Plot
- 12.5 Bubble Plot
- 12.6 Contour Plot
- 12.7 Histogram
- 12.8 Line Plot
- 12.9 Pie Chart
- 12.10 Scatter Plot
- 12.11 Scatter Plot Matrix

13 Compare groups and fit models
- 13.1 One-Way ANOVA
- 13.2 t Test
- 13.3 Linear Models
- 13.4 Linear Regression
- 13.5 Logistic Regression
- 13.6 Generalized Linear Models
- 13.7 Nonlinear Regression
- 13.8 Correlations
- 13.9 Principal Components
- 13.10 Cluster Analysis

14 Check patterns and work with time
- 14.1 CDF Plots
- 14.2 Probability Plots
- 14.3 Control Charts
- 14.4 ARIMA / Basic Forecasting

15 Time until an event
15.1 Life Tables
15.2 Proportional Hazards
16 Run steps conditionally or repeatedly
16.1 Branch
16.2 Loop
16.3 Choose a loop source
16.4 Start small
17 Keep a useful log
17.1 Choose how much to record
17.2 Customize a task's message
17.3 Add a message inside a Script
17.4 Export Log
17.5 Put the log in a report
17.6 Read dates as well as messages
17.7 Use startup logs for startup problems
18 Share reports and files
18.1 Report
18.2 Choose HTML or Word
18.3 Pick a content style
18.4 Export
18.5 Choose an export format
18.6 Check a finished delivery
19 Optional: try the R Console
19.1 Open the console and run a command
19.2 Read a command before running it
19.3 Find and reuse a command
19.4 Clear, Stop, and Reset
19.5 Keep useful code in a task
20 Optional: export the flow as R code
20.1 Share code that someone can run
21 Explain and review your work
21.1 Add notes at decision points
21.2 Check before sharing
21.3 Find something quickly
21.4 A simple review checklist
22 Make the app comfortable to use
22.1 Display and navigation
22.2 Saving and confirmation choices
22.3 Trust and private information
23 Keep work running smoothly
23.1 When editing feels slow
24 Troubleshooting
24.1 The app seems not to open
24.2 The app disconnects or the R server stops
24.3 A button or menu item is disabled
24.4 A file will not import
24.5 A table or column is missing from a choice list
24.6 A result is old, empty, or different from the preview
24.7 A model fails or looks strange
24.8 A report is missing a chart
24.9 An exported file is missing or will not update
24.10 A log seems empty
24.11 Ask for help
25 Optional: install with R
25.1 Step 1: install R from its public website
25.2 Step 2: install the app from GitHub
25.3 Step 3: add the features you need
25.4 Step 4: open the app
25.5 Open it next time
25.6 If package installation fails
26 Choose an example project
26.1 Find an example by task
26.2 A simple learning route
26.3 Before you rerun an example
26.4 Iris exploration
26.5 Air quality - clean and compare
26.6 Fuel economy model
26.7 Cars - prepare an analysis table
26.8 US states - import, join and filter
26.9 Iris - reshape and compare
26.10 Cars - rank, scale and sample
26.11 Tooth growth - descriptive analysis
26.12 Flowers - categorical chart gallery
26.13 Flowers - numeric chart gallery
26.14 Nottingham temperatures - time charts
26.15 Plant growth - compare experimental groups
26.16 Cars - binary outcome model
26.17 Loom breaks - a count model
26.18 Puromycin - fit a nonlinear curve
26.19 Flowers - multivariate structure
26.20 Lung cancer - survival analysis
26.21 Air passengers - forecast a monthly series
26.22 Nile flow - distribution and stability
26.23 Flowers - conditional quality gate
26.24 Flowers - repeat by species
26.25 Cars - a report with an execution audit
27 Task finder
28 Shortcuts and everyday terms
28.1 Useful Windows desktop shortcuts
28.2 A few more terms
28.3 Public downloads and help

1 Welcome to Individual Guide

You can use Individual Guide without knowing R. The app helps you bring in data, clean it, make charts, and share results. You choose settings in forms. The app does the calculations for you.

Think of a project as a recipe. Your data are the ingredients. Each **task** is one step, such as sorting rows or drawing a chart. A **flow**, also called a pipeline, puts those steps together. You can save the recipe and use it again.

This guide covers **Individual Guide 0.4.0**, with **58 task types** and **22 example projects**. Start with installation and the first project. After that, read the task pages you need. You do not need to read the book from start to finish.

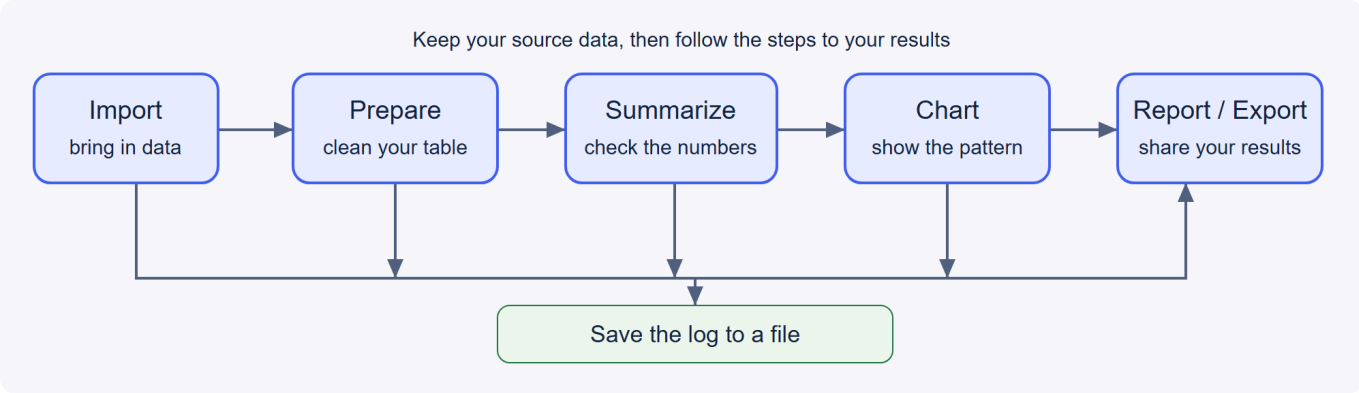
1.1 Find your starting point

What you want to do	Where to go
Install the Windows app	Installation and first launch
Try the app with ready-made data	Your first project
Bring in your own spreadsheet or CSV	Importing your data
Find a task and a matching example	Task finder
Make a report with a log	Reports and files
Fix a problem	Troubleshooting
Run the app through R instead	Optional: install with R

1.2 Six words you will see often

Word	Meaning in the app
Project	Your saved work: data, tasks, results, and notes
Table	Data arranged in rows and columns, like a spreadsheet
Task	One step that does a job
Flow	A group of connected tasks
Output	A result from a task, such as a table or chart
Log	A record of what happened when tasks ran

For example, a flower project can start with measurements, calculate a mean, draw a chart, and make a report. The arrows show which steps need results from earlier steps.



A flow takes data through tasks to results, reports, and a saved log.

1.3 How to follow this guide

Bold words, such as **Run**, name buttons or choices. **File > Save As** means open the File menu, then choose Save As. Text such as `flowers` is a name to type exactly.

The Windows app has menus across the top. If you open the app through R in a web browser, use the buttons inside the app for opening and saving projects. Some desktop features, such as restarting the R server from a menu, are only in the Windows app.

The HTML book works offline after you download it. Use **Find a chapter or task** in its contents, or press Ctrl+F to search the page. The PDF is useful for printing. EPUB works in an e-reader, and Word lets you add your own notes.

2 Installation and first launch

For most Windows users, the Setup installer is the easiest choice. It includes R and the extra software the app needs. **You do not need to install R or RStudio separately.**

2.1 Download the public Windows release

You need a 64-bit Windows 10 or Windows 11 computer. Allow about 1.5 GB for the program, plus space for your projects. You need an internet connection to download Setup. The installed app and its examples can then work offline. Tasks that use an online database still need that connection.

1. Open the public releases page.
2. Under **Assets**, download **IndividualGuide-0.4.0-Setup-Windows-x64.exe** for this edition. If a newer release is shown, read its notes before choosing it. Find this book's matching installer on the 0.4.0 release page.
3. Open the downloaded file. You do not need the files called **Source code** to use the desktop app.
4. Close any running copy of Individual Guide. Double-click Setup and follow its instructions. The usual installation is for your own Windows account.
5. Open **Individual Guide** from the Start menu or desktop shortcut.

The installer is unsigned. Windows may show an unknown-publisher warning. Check that you downloaded it from **woodtho/IndividualGuide** before deciding whether to continue. If your work or school computer blocks it, ask your support team for help.

2.2 Open the app each day

Use the Start menu or desktop shortcut. Setup installs or updates the program; you do not need to run Setup each time.

The first launch may take longer than later launches. Wait while the startup screen shows its progress. Once the workspace appears, start a new project or open an example from Help.

If startup stops with an error, choose **Open startup log**. See Troubleshooting for the next step. Repeatedly opening more copies usually does not help.

2.3 Update the app

1. Save your project and close the app.
2. Visit the public releases page.
3. Download and run the newer Setup installer.
4. Open the app from its shortcut. Check **Help > About** to see the installed version.

The app does not update itself. Updating keeps saved projects and settings, but keep your own backup of important project files. Uninstalling through Windows removes the program and keeps separately stored user data.

2.4 Before your first project

Create a folder you can find again, such as **Documents > Individual Guide Projects**. Use it for saved projects and files you export. Give each project a clear name, such as `flower-practice.igpx`.

The file ending **.igpx** means an Individual Guide project. Open it through **File > Open project**, or double-click it in Windows after installing the app.

You are ready to try your first project. The optional R installation steps are near the end of this guide; the Windows app does not need them.

3 Your first project

Start with an example that already works. You will open flower data, find a chart, check a report, and save your own copy. No typing of R code is needed for this first part.

3.1 Open the flower example

1. Open **Help** in the app and find **Example projects**. In the desktop app, **Help > Application help** opens app help.
2. Search for **Iris exploration**.
3. Choose **Open example**. If you already have unsaved work, save it before replacing the current project.
4. Choose **File > Save As**. Save the project as `flower-practice.igpx` in your projects folder. In a browser, use the app's Save As button and file picker.

Iris is a kind of flower. The example has measurements from **150 flowers** in **three species**, with **50 flowers per species**. Each row is one flower. The four measurement columns use centimetres. `Species` names the flower's group.

Opening an example gives you a separate working copy. You can change it without changing the example that ships with the app. Save your copy if you want to keep those changes.

3.2 Look around before running anything

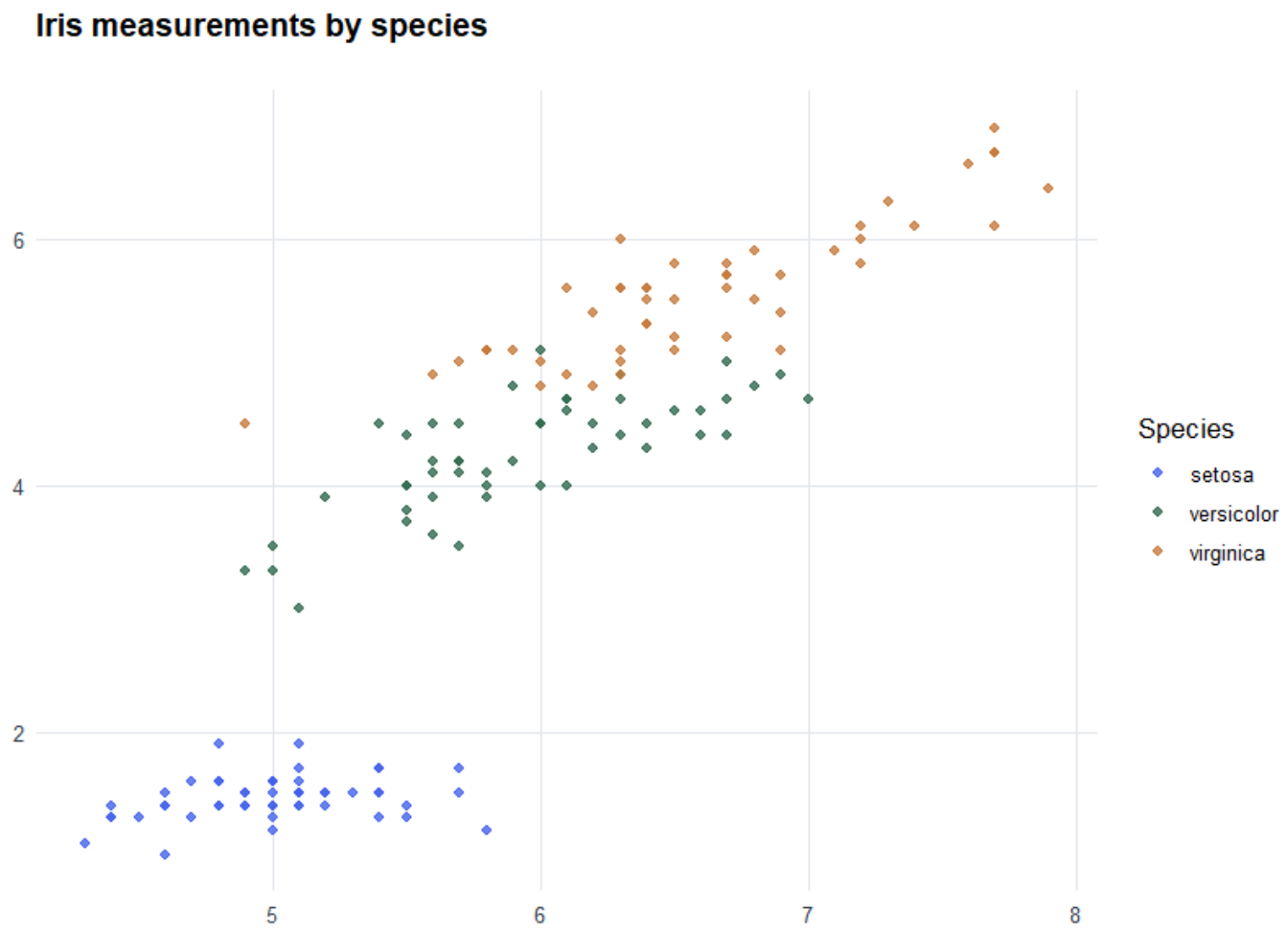
The left side lists the project's tasks and data. The **Process Flow** view shows connected steps. Select the source task and read its notes. Open its flower table and look at the column names.

Find **Summary Statistics**, **Summary Tables**, and **Scatter Plot** in the flow. Open each task's Results. The example already contains results, so you can explore before running it again.

For the species summary, the mean petal lengths should be about:

Species	Mean petal length, in centimetres
setosa	1.462
versicolor	4.260
virginica	5.552

You do not need to memorize these values. They give you a way to check that you are viewing the right table and settings.



The flower example compares sepal length and petal length. Colour shows the species.

3.3 Make one small change

1. Open the **Scatter Plot** task.
2. Find the Y column setting. Change it from `Petal.Length` to `Petal.Width`.
3. Update the chart title so it describes petal width.
4. Click **Run** in the task header. Open Results and check the new chart.
5. Run the **Report** task again so its chart matches your change.
6. Save your project.

Changing a setting does not update a saved report by itself. Run the changed task, then the later tasks that use its result. If you want to run everything again, use the flow's play button. Before doing that, check the Export tasks' folders and Overwrite settings.

3.4 Find the report and log

Open **Report**, then open or download the finished document. Check its headings, tables, and chart. In a Report task, **Appearance > Include pipeline log in the report** adds a section called **Pipeline execution log**. Turn it on and run the report again if needed.

Open **Export Log** to see how the example writes its log to a file. Before running this task, choose a destination you can find. Logs can include earlier runs as well as the latest one. Dates and times help you tell them apart.

3.5 Build a small flow yourself

This optional exercise builds the same kinds of results from a new project. It includes one short piece of R code that you can copy exactly. You do not need to understand the code to finish the exercise. Stay with the ready-made examples if you prefer.

Choose **File > New project** and save it as `first-iris.igpx`. Use **Add Task** to find each task below. Task labels help you recognize a step. Output names tell later steps which result to use, so type those names carefully.

3.5.1 Step 1: load the practice data

Add **Script** from General. In its outputs, add a **table** named `flowers`. Paste this into the Script editor, not the R Console:

```
flowers <- datasets::iris
log_event(sprintf("Loaded %d flowers", nrow(flowers)), "INFO")
```

Click **Run**. Open `flowers` in the project and check for 150 rows and five columns. If it is missing, make sure you added the table output named `flowers`. The name in the output list must match the name in the copied code.

3.5.2 Step 2: summarize the measurements

Add **Summary Statistics** from Describe. Select `flowers` as the input. Choose `Sepal.Length`, `Sepal.Width`, `Petal.Length`, and `Petal.Width`. Name the output `measurement_summary`, then run it.

Check the mean for `Sepal.Length`. It should be about **5.8433**. A mean is the total divided by the number of values. The app may show fewer decimal places.

3.5.3 Step 3: compare the three species

Add **Summary Tables**. Choose these settings:

Setting	Choice
Input table	`flowers`
Row variable	`Species`
Value variable	`Petal.Length`
Statistic	Mean
Output name	`species_summary`

Run it and compare the three results with the table earlier in this chapter.

3.5.4 Step 4: draw the chart

Add **Scatter Plot**. Select `flowers`, set X to `Sepal.Length`, Y to `Petal.Length`, and the colour or group to `Species`. Set the title to **Iris measurements by species**.

In the output controls, enable a **static image** and name it `iris_scatter_image`. Run the task. A static image is a still chart that you can put in a report or export as PNG.

3.5.5 Step 5: save a table and chart as files

Add **Export**. Select `species_summary`, choose CSV, choose your projects folder, and enter `species-summary.csv`. Run the task and check that the file appears in that folder.

Add another Export task for `iris_scatter_image`. Choose PNG, set a filename of `iris-scatter.png`, and use the same folder. Set DPI to 150 for this exercise. DPI controls how many dots are used per inch. Run it and open the image.

If a file already exists, the task may ask you to allow Overwrite. Turn that on only when you want to replace that file. An exported file is separate from your saved project.

3.5.6 Step 6: make a report

Add **Report**. Under Content, add `measurement_summary`, `species_summary`, and `iris_scatter_image`. Give the items clear headings. Under Appearance, choose HTML and set the title to **First Iris report**.

Turn on **Include pipeline log in the report** and leave its minimum level at INFO. Run the task. Open the report and check for the two tables, the chart, and the Pipeline execution log section.

3.5.7 Step 7: save the log

Add **Export Log**. Select the preceding tasks, including Report, so it waits for them. Choose CSV and a full destination such as `iris-log.csv` inside your projects folder. Use the folder picker or enter the full path shown for your chosen folder.

Run the flow. Check the log file for **Loaded 150 flowers** and the Report completion entry. The report's own log section is made before the report finishes. The final Export Log can record that later completion.

3.5.8 Step 8: close and reopen your work

Save `first-iris.igpx`, close the project, and open it again. Check that its table, chart, and report are still there. You have made a reusable flow with data, results, a report, and a log.

4 Find your way around

The project list, flow diagram, and task panels show the same work in different ways. Use whichever makes the next step easiest to find.

4.1 Open a task or result

Select a task in the project list or flow to open its panel. Its settings control what it does. Results show what happened when it last ran. The Log explains messages and errors. Code shows the R instructions behind the task; reading it is optional.

Closing a task tab hides that panel. It does not delete the task. Use the task's delete action only when you want to remove the step itself.

If the diagram becomes crowded, open **Project tools > Task list**. Search for the step you need and open it from the list. Use **Fit view** to bring the whole flow back into view.

4.2 Understand the arrows

A later task needs the right input name. For example, a summary that uses `flowers` must wait for the task that creates `flowers`. The app draws the connection from those choices.

Dragging a task to a new place changes the layout. It does not change which tasks run first. To change that, edit its inputs or its branch and loop settings.

4.3 Choose useful names

Use a task label that explains the job, such as **Remove repeated customer rows**. Use a short, unique output name, such as `customers_clean`. Avoid giving two outputs the same name.

Changing an output name can affect tasks that use it. Check those tasks' inputs and run the flow again after a rename.

5 Save, reopen, and share projects

5.1 Save your project

Use **File > Save** or Ctrl+S in the desktop app. The first save asks for a name and folder. Use **Save As** to keep a separate copy before a large change.

In a web browser, use the app's Save controls. Follow its file picker or download prompt and check where the file was saved.

A saved project keeps tasks, settings, available results, notes, and retained task logs. It does not automatically copy every outside file used by your tasks. Keep important input files in a stable folder.

5.2 Move work to another computer

Use the project's files and portability tools to make a **portable project** when you need to bundle supported input files. Check the listed files before exporting. Open the copy on the other computer and check its inputs before running it.

Database connections still need access to the database. A Script may refer to a file that the portable export cannot find on its own. If so, include that file separately and update the path. A path is the address of a file or folder on your computer.

Reports and files that you already exported are separate copies. Saving the project again does not update those copies. Run the relevant Report or Export task when results change.

5.3 Recover interrupted work

The app keeps recovery information for unsaved work. After a crash or restart, it may offer a recovery copy. Read the prompt, recover the work you need, then save it with a clear filename.

Recovery helps after a problem. Keep using Save during normal work. If you need an earlier saved copy, open **Saved versions** from the project tools or File menu. Check the version date before restoring it.

5.4 Share a project with care

A project can contain data, paths, logs, and R code. Share only information you mean to share. Open projects from people or sources you trust, because running tasks can run code and write files.

Before sending a project, save it, check its report, and try opening your saved copy. If the recipient only needs the results, send a report or CSV instead of the full project.

6 Run tasks and check results

Start with one task while you are learning. Once each step works, run the whole flow.

6.1 Run, Preview, and Save do different jobs

Action	What it does
Preview	Lets you inspect a trial result where the task supports it
Run or Run & Save	Runs the task and keeps its result in the project
Run flow	Runs connected tasks in the active flow
Save project	Writes your project to its .igpx file

After a Preview, use Run to keep the result. After running, use Save to keep the project on disk. A Script can write outside files even during a preview, so preview only code you trust.

6.2 Check the task status

A successful status means the task finished. Open its result to make sure it answered your question. An error means it needs attention. Open its Log and read the first useful error message.

A stale result means an input or setting has changed since that result was made. Run the affected task and the later tasks that use it. An old report may still open while it waits to be rebuilt.

A skipped task may belong to a branch that was not chosen. A waiting task may need an earlier task to finish. Check inputs and branch settings before treating either state as a failure.

6.3 Run the right part of a project

Use the flow's play button to run the active flow. Check which flow is selected if your project has more than one. Right-click a task for **Run to here** or **Run from here** when those actions are available. These follow task connections, rather than the positions of shapes on the screen.

Use **Validate active flow** to check for missing inputs and other setup problems before a long run. Validation checks the setup. It cannot prove that your data or conclusions are correct.

6.4 Stop a run

Use **Stop** or **Task > Stop run** in the desktop app. Some work may need to finish its current step before it can stop. Wait for the status to change before starting another run.

Stopping does not undo files that tasks already wrote. Check export folders before running again. If you want to avoid replacing files, use new filenames or turn off Overwrite.

7 Importing your data

For a first import, use a small CSV or Excel sheet with one header row. Give every column a name. Keep titles, footnotes, and totals outside the data area.

7.1 Import Data

Use **Import Data** to bring a file or database table into the project.

1. Add the task from Data. Choose the file type and select your file.
2. For CSV or other text files, check the separator, header, and text encoding. A separator is the character between columns, often a comma or tab.
3. For Excel, choose the sheet and any row or range settings you need.
4. Check the preview. Each field should be in its own column, with the right names and values.
5. Choose a clear output name, such as `sales_raw`, and run the task.

Open the new table and check its row count. Check dates, missing values, and identifiers such as postal codes. If an ID starts with zero, keep it as text so the zero is not lost.

If all values land in one column, check the separator. If the names become the first data row, check the header setting. If letters look wrong, try the file's text encoding. The **US states - import, join and filter** example uses a real bundled CSV that you can inspect without finding your own file.

7.2 Work with a database

Choose the database source in Import Data and select or set up its connection. You may need an address, database name, account, and a driver from your database provider. Ask the person who manages the database for these details.

Start with a small table or query. A live database result may still need its connection when later tasks run. If you need a fixed copy for a report, make sure the data have been loaded into the project and record when you took the copy.

The Windows app includes its normal app packages. An R-only installation may need a database package and a separate system driver. The optional R setup explains where to get packages. Do not put passwords in a report, log message, or shared Script.

8 Select, join, and calculate data

These tasks help you choose the rows and columns you need. Run the input task first so column lists can show its results.

8.1 Query Builder

Use **Query Builder** when you need to choose columns, join tables, filter rows, or make grouped totals in one task.

Select the input tables, then choose the columns to keep. Add a join only when you need columns from another table. A join matches rows using a shared key, such as a customer number. Pick the key on both sides and choose the join type. A left join keeps the rows from the first table and adds matches from the second.

Preview the result and check its row count. When it looks right, run the task to keep the result. If a key appears several times in both tables, a join can produce more rows than you expected. Check keys before calculating totals.

For grouped results, choose the group columns and a calculation such as sum or mean. Filters before grouping choose input rows. A filter on the finished group result chooses which totals remain. Start with the **US states** example to see a join and a filter together.

8.2 Filter and Sort

Use **Filter and Sort** to keep matching rows and put them in order.

Choose the input table. Add a rule by selecting a column, a comparison, and a value. For example, a numeric amount greater than 100 keeps rows above that amount. Check whether several rules should all match or whether any one may match.

Choose one or more sort columns and their order. Ascending goes from smaller to larger, or A to Z. Descending reverses that order. Run the task with a new output name and check a few rows at the top and bottom.

Missing values need their own rule. A missing value is an unknown entry, not the same as zero or an empty word. Use the missing-value choice when you mean to select those rows. The **US states** example shows how filtering changes the final table.

8.3 Derive Columns

Use **Derive Columns** to add a calculated column, such as a total or a value converted to new units.

Choose the input, enter a new column name, and enter its formula. A simple formula can use existing column names and arithmetic. For example, if the input has columns named `price` and `quantity`, `price * quantity` calculates their product for each row.

Start with one new column. Run the task and check a few values by hand. Then add other calculations if needed. Keep units clear: changing the column name alone does not convert centimetres to metres.

Formulas use R expressions, so names and punctuation matter. Copy a working formula from a matching example when you are unsure. The **Cars - prepare an analysis table** example shows a calculated field. You can skip this task until you need a calculation that simpler tasks do not provide.

9 Optional: saved code steps

9.1 Script

Use **Script** when you have trusted R code to run as part of a flow. You can complete many projects using the other task forms instead.

Select the input tables the code needs. Paste code into the Script editor. In the output list, declare each result you want to keep, using exactly the name used by the code. For a data table, choose a table output. A model or other non-table result needs an object output.

Run the task and check Results and Log. If a result is missing, first compare its declared name with the code. Names are case-sensitive: `Flowers` and `flowers` are different.

The first-project exercise includes a two-line Script you can copy. The examples also contain prepared Scripts with notes. Read those notes before changing their code. Use the R Console for a quick trial, then keep repeatable code in a Script task. A console command on its own does not become a saved step in the flow.

10 Clean and reshape your data

Keep an unchanged input table, then give each cleaned result a new name. This makes it easier to check what changed. After each task, compare row counts, column names, and a few values.

10.1 Append Table

Use **Append Table** to put rows from one table below rows from another. For example, you could combine January sales with February sales.

Choose the tables and check that matching columns mean the same thing. A column called `amount` should use the same units in both. Choose the output name and run the task. Check the final row count and any blank values caused by columns that were only present in one input.

Appending adds rows. It does not match customer details across tables; use Query Builder for that. Open **Iris - reshape and compare** to see two species subsets combined into 100 rows.

10.2 Compare Data

Use **Compare Data** to check two tables for differences. It is useful after cleaning, importing a new extract, or reshaping data and changing it back.

Select the two tables. Review the comparison settings, including any key or row-matching choices offered. Run the task and read the differences rather than only checking whether it finished. A changed row order or column type can matter even when the displayed values look similar.

If the tables have different row counts, first find why. A filter may have removed rows on purpose. The **Iris - reshape and compare** example compares a table before and after a reshape. Use its notes to see which differences are expected.

10.3 Stack Columns

Use **Stack Columns** to turn several measurement columns into rows. This is often called making data long.

Choose the input, the columns to stack, and the columns to keep as identifiers. Set names for the new measurement-name and measurement-value columns. Run the task and check that each original observation can still be identified.

In **Iris - reshape and compare**, four measurements for each of 150 flowers become 600 rows. A flower identifier shows which rows belong together. Keep that identifier if you may want to turn the table back into columns later. Extra rows are expected here; they do not mean you have more flowers.

10.4 Split Columns

Use **Split Columns** to turn long-form values back into separate columns. This is often called making data wide. Here, Split Columns means reshaping a table; check its settings rather than assuming it splits words at commas.

Choose the identifier columns, the column whose values will become column names, and the column containing the measurements. Run it and check the output names and row count.

Each identifier and new-column pair should describe one value. If several rows claim the same place, inspect those rows before continuing. They may need a summary first. The **Iris - reshape and compare** example shows a full long-to-wide round trip using a flower identifier.

10.5 Transpose

Use **Transpose** to turn rows into columns and columns into rows. It is most useful for a small table that needs a different display shape.

Select the input and the columns you want to transpose. Check naming choices, run the task, and inspect the new table. Make sure the new column headings still tell the reader what each value means.

Start with a few rows. Transposing a large table can create a very wide result that is hard to view or export. Mixing text and numeric values may also change how values are stored. The **Iris - reshape and compare** example deliberately transposes a small numeric subset.

10.6 Rank

Use **Rank** to add positions such as first, second, and third based on a column’s values.

Choose the input and ranking column. Select the order and the method for ties. Ties occur when two rows have equal values. Different methods can give those rows the same rank or leave gaps after them.

Run the task with a new output name. Check the highest and lowest values and at least one tie. A rank is a position, not the original measurement. Keep the original column if later work needs its values. The **Cars - rank, scale and sample** example shows ranked fuel economy.

10.7 Standardize Data

Use **Standardize Data** when numeric columns need a common scale before a later analysis. For example, weight and length can have very different ranges.

Choose the numeric columns and the scaling method. Read the method’s label before running it. A common method subtracts the mean and divides by the standard deviation, which measures spread. The result then uses a new scale instead of the original units.

Run the task and check the output. Keep the original table for reference. A constant column has no spread, so it needs attention before some scaling methods work. Open **Cars - rank, scale and sample** for an example. Scaling is a choice for a later analysis, not a routine fix for every dataset.

10.8 Random Sample

Use **Random Sample** to choose part of a table without simply taking the first rows. It can help you try a flow on less data.

Choose the input, sample size, and any replacement setting. With replacement means the same row may be picked more than once. Set a **seed** if you want to repeat the same random choice with the same data and order.

Run the task and check the sample count. Do not treat a small practice sample as if it were the full dataset. The **Cars - rank, scale and sample** example picks ten rows with seed 2026. Try a different seed in a saved copy and compare which rows were chosen.

10.9 Remove Duplicates

Use **Remove Duplicates** to remove repeated rows based on selected columns.

Choose the columns that define a duplicate. All columns means only fully repeated rows match. A smaller key, such as customer number, can also match rows whose other values differ. Check which row will be kept before running.

Run the task, then compare the row counts before and after. Inspect a few removed cases if you are unsure. Two rows can look similar and still describe separate events, so do not remove them just to make the table smaller. **Cars - prepare an analysis table** adds one duplicate on purpose: its 33 source rows become 32 after this task.

10.10 Keep / Drop Columns

Use **Keep / Drop Columns** to choose which columns remain in a table.

Select the input, choose Keep or Drop, then select the columns. Keep retains only those selected. Drop removes the selected columns. Give the result a new name and run it.

Open the result and check its column names. Keep an ID column if later joins or checks need it. Removing a column from this output does not remove it from the unchanged source table. If a later task says a column is missing, check whether this step removed it. The **Cars - prepare an analysis table** example shows a smaller analysis table.

10.11 Rename Columns

Use **Rename Columns** to make headings easier to understand.

Choose the input and add each old name with its new name. Use unique names. Short names with letters, numbers, and underscores are easy to use in later formulas. Run the task and check the result's headings.

Renaming changes the label, not the values or their units. A weight column does not become kilograms because you call it `weight_kg`. Add a calculation when you need a real conversion. Check later tasks after renaming because their selected columns may need to change too. **Cars - prepare an analysis table**

includes a rename you can follow.

10.12 First / Last N Rows

Use **First / Last N Rows** to keep a set number of rows from one end of a table.

Choose the input, First or Last, and the number of rows. Run it and check the result. The table's current order decides which rows count as first or last. Sort first if you want, for example, the ten largest sales.

This task is useful for a quick check or a short display. It is not a random sample. If the first rows all come from one date or group, they may give a poor picture of the whole table. See **Cars - prepare an analysis table** for a small selection.

10.13 Replace Missing Values

Use **Replace Missing Values** when you have chosen a reason to fill unknown entries.

Select the input, the columns, and a replacement method. Depending on the column and method, you may enter a value or choose a calculated value such as a mean. Run it and count how many missing entries remain.

Keep the raw data and note the method you chose. Filling a gap does not recover the unknown original value. It can change summaries and models. **Air quality - clean and compare** uses mean replacement to teach the controls. That example is a practice choice, not a recommendation to fill every dataset that way. You can inspect the missing-value audit before changing anything.

10.14 Recode Values

Use **Recode Values** to replace selected values with new values or labels. For example, you might change 0 and 1 to **Automatic** and **Manual** in a display column.

Choose the column and add the old-to-new mappings. Check what should happen to values you did not list. Run the task and count each category to make sure no group was lost or changed by accident.

Keep the original codes when you may need them for a later model. A model may expect numbers even when a chart is easier to read with words. **Cars - prepare an analysis table** shows labelled transmission values and how they relate to the original codes.

10.15 Change Column Types

Use **Change Column Types** when values are stored in the wrong kind of column. Common types are text, number, date, and factor. A factor stores a set of categories.

Select the column and its intended type. For dates, check the date format. Run the task, then inspect both the type and the values. Check for new missing values, which can mean some entries could not be converted.

Keep identifiers as text when leading zeros matter. Use numeric types for real measurements and categories for group labels. **Cars - prepare an analysis table** changes a category's type. If a column is absent from a chart or model list, its current type is one of the first things to check.

11 Get to know your data

Look at your data before fitting a model. Check how many rows you have, which values are missing, and how each column is stored. These tasks help you spot surprises early.

11.1 Characterize Data

Use **Characterize Data** for an overview of a table's columns.

Choose the input table and the columns to inspect, then run the task. Review the column types, counts, and missing-value information. Check that measurements are numeric, group labels are categories or text, and date columns contain dates.

This is a good first step after Import Data. If a number column is stored as text, look for entries such as currency symbols or words before trying to convert it. Use Change Column Types once you know the intended meaning. The **Tooth growth - descriptive analysis** example starts with checks that help you understand the data before summarizing it.

11.2 List Data

Use **List Data** to make a readable view of selected rows and columns.

Choose the input, the columns, and any row limit or display settings. Run the task and read a few records. Include a useful identifier so you can trace an unusual value back to its source row.

A short listing helps you spot misspelled categories, unexpected dates, and values in the wrong units. Check the displayed row count or limit before assuming the listing shows every record. Keep your full input table for calculations. See **Tooth growth - descriptive analysis** for a listing alongside summary tasks. Use Export when you need to deliver a data file rather than a view inside the project.

11.3 One-Way Frequencies

Use **One-Way Frequencies** to count how often each category appears in one column.

Choose the input and category column. Set any missing-value and percentage options, then run it. Check the counts and their total. If a category seems to be missing, inspect blank values and spelling in the source table.

A count tells you how many records are in a group. A percentage also needs a denominator, such as all rows or only rows with known values. Check which rows the task includes before reporting percentages. **Tooth growth - descriptive analysis** counts supplement groups. You can also count the Iris species to check that each has 50 flowers.

11.4 Summary Statistics

Use **Summary Statistics** for numbers such as mean, median, minimum, maximum, and spread.

Choose the input and numeric columns, then run it. Read the number of usable values as well as the summary. A mean based on ten known values does not describe fifty complete observations when forty entries are missing.

The mean is the average. The median is the middle value after sorting. A few very large values can pull the mean away from the median, so comparing both can help you spot an uneven pattern. **Iris exploration** gives you a simple check: mean sepal length is about 5.8433 centimetres. Keep units in the headings when you put the results in a report.

11.5 Summary Tables

Use **Summary Tables** to calculate a value for each group, such as mean sales by region.

Choose the input, a row or group variable, a value variable, and a statistic. For example, use `Species` for groups, `Petal.Length` for values, and mean for the statistic. Choose an output name and run the task.

Check how many groups appear and whether their labels match the source. Groups with missing measurements may have different numbers of usable rows. Read those counts before comparing means. **Iris exploration** shows three species means, and **Tooth growth - descriptive analysis** shows a grouped summary for another dataset. A grouped table is also a useful input for a report or bar chart.

11.6 Table Analysis

Use **Table Analysis** to compare two category columns in a cross-table. Each cell counts rows that match one category from each column.

Choose the row category and column category. Run the task and check the counts before reading any test results. Review whether displayed percentages are based on rows, columns, or the whole table.

If you enable a chi-square test, it checks for evidence of a relationship between the categories. Small or empty cells can make that test unsuitable. A small p-value does not tell you how large or useful the relationship is. **Tooth growth - descriptive analysis** cross-tabulates supplement and dose. It helps you see how the observations are divided among the study groups.

11.7 Distribution Analysis

Use **Distribution Analysis** to examine how a numeric column’s values are spread out.

Choose the input and measurement column. Select the summaries or displays you need, then run it. Look for a long tail, several peaks, extreme values, and missing entries. A long tail means a few values extend far beyond most of the data.

Use what you see to guide your next question. Do not delete a large value just because it looks unusual; check whether it is a mistake or a real observation. **Tooth growth - descriptive analysis** includes this task beside counts and grouped summaries. Together, those views help you understand both the measurements and the groups that produced them.

12 Make charts people can read

Choose a chart based on the question you want to answer. Start with simple settings, then add a clear title, axis labels, and units.

The **X** setting usually controls the horizontal axis. **Y** controls the vertical axis. Group or colour separates categories. After running, check both the chart and the number of rows it uses.

12.1 Keep the output you need

Chart tasks can offer plotting data, an interactive chart, and a static image. An interactive chart may show details when you hover. A static image is a still chart that works well in Word reports and image exports.

Enable the static image output if you want to export PNG. Then use an **Export** task to write the file. Making a chart in the project does not by itself write a picture into your chosen folder.

Some chart views and chart data outputs show only the first 1,000 rows. Check the task's row limit before using a chart output as if it were the complete source table. Use a summary or sample on purpose when there are too many points to read.

12.2 Bar Chart

Use **Bar Chart** to compare category counts or a selected numeric summary.

Choose the category for X. If the chart mode asks for a value or summary, choose those too. Add a group only if it helps the comparison. Run the task and check that the axis describes what the bar height means: a count, sum, and mean answer different questions.

Keep category names readable. If the labels overlap, reduce the number of categories or change the layout. **Flowers - categorical chart gallery** counts the three species. Each bar should represent 50 flowers. Compare it with the frequency table if you are unsure what the chart is counting.

12.3 Bar-Line Chart

Use **Bar-Line Chart** to show bars and a line on the same category or time axis.

Choose the shared X column, then the values for the bars and line. Check axis labels and any second-axis settings. Run the chart and make sure the legend clearly identifies each part.

If the two values use different units or scales, explain both. Similar-looking heights on two axes do not mean equal values. Avoid using the chart to imply a relationship just because the shapes line up. **Nottingham temperatures - time charts** compares annual averages with observed ranges. The range is a difference between observed values, not a measure of uncertainty around the average.

12.4 Box Plot

Use **Box Plot** to compare the spread of a numeric measurement across groups.

Choose the group for X and the measurement for Y. Run the task. The line inside the box marks the median, the middle value. The box covers the middle half of the data. Points beyond the whiskers highlight values that may deserve a closer look.

Read the plot alongside group counts. A small group gives less information than a large one, even if their boxes look similar. Check unusual points before deciding they are errors. **Flowers - categorical chart gallery** compares flower measurements by species. Use Summary Tables if you also need exact group values in a report.

12.5 Bubble Plot

Use **Bubble Plot** to show two numeric measurements, with bubble size showing a third.

Choose X, Y, and a numeric size column. Add colour or group if useful, then run the task. Check that the size legend explains what larger bubbles mean. Keep the units clear for all three measurements.

Overlapping bubbles can hide smaller ones. Try less data or a simpler Scatter Plot if the picture is hard to read. Use this task when size needs to change visibly; a Size field shared with other chart forms may not have that effect in Scatter Plot. **Flowers - numeric chart gallery** lets you compare both charts using the same observations.

12.6 Contour Plot

Use **Contour Plot** to see where pairs of numeric values are concentrated.

Choose the two numeric columns for X and Y, then run the chart. Read the contour lines or regions as a guide to where more observations lie. Compare the result with a scatter plot of the same columns.

The smooth shape depends on the data and chart settings. It does not show a measured boundary around a real group. A small sample can give a pattern that looks more certain than it is. **Flowers - numeric chart gallery** includes contours beside the individual flower points, which makes it easier to see what the smoothing adds and what it leaves out.

12.7 Histogram

Use **Histogram** to show how one numeric measurement is distributed.

Choose the numeric column and a bin setting. A bin is a range of values counted together. Run the task and check the horizontal axis, the counts, and the number of missing values.

Try a few sensible bin choices. Very wide bins can hide patterns, while very narrow bins can make random variation look important. A histogram’s bars represent numeric ranges; a Bar Chart usually represents separate categories. **Flowers - numeric chart gallery** includes a histogram you can compare with its Summary Statistics. Use both when you want a picture of the spread and exact values.

12.8 Line Plot

Use **Line Plot** to show change across an ordered column, often time.

Choose the date or ordered value for X and a measurement for Y. Add a group if you need separate lines. When several rows share the same time, decide whether you need to summarize them first. Check the order, then run the task.

A line joins observations. It does not prove what happened between them, and it is not a forecast. Check gaps and the time range so readers can see where data are missing. **Nottingham temperatures - time charts** shows 240 monthly observations. Read the date column and units before comparing years or describing a seasonal pattern.

12.9 Pie Chart

Use **Pie Chart** for a simple view of category shares when there are only a few groups.

Choose the category column and run the task. Check each category's count and share. In this task, the slices count observations by category; do not assume a numeric amount column supplies slice weights.

Use a frequency table to confirm the totals. Many small slices are hard to compare, so a Bar Chart may be easier to read when you have many groups. **Flowers - categorical chart gallery** uses the three Iris species. Its slices should be equal because each species has 50 observations. Changing colour affects the display, not the counts behind it.

12.10 Scatter Plot

Use **Scatter Plot** to look at the relationship between two numeric columns.

Choose X and Y, then add colour or group if it helps. Run the chart and check the labels and units. Each visible point represents an observation, although points can overlap.

Look for a trend, curved pattern, separate groups, and unusual points. A visible trend does not prove that one measurement causes the other. Use Bubble Plot if you need a third measurement to control point size. **Iris exploration** compares sepal length with petal length by species. Try changing Y to petal width in a saved copy, then rerun the report so it shows your new chart.

12.11 Scatter Plot Matrix

Use **Scatter Plot Matrix** to compare several numeric columns in one grid.

Choose a small set of numeric columns and any grouping setting, then run it. Each panel compares a pair of columns. Read its row and column labels to know which measurements are shown.

Start with three or four columns. A large grid becomes hard to read, and display limits may show only part of a large dataset. If one pair is interesting, open a separate Scatter Plot for a clearer view. **Flowers - numeric chart gallery** compares the four Iris measurements. Use its matrix to choose a relationship to inspect, rather than trying to explain every panel at once.

13 Compare groups and fit models

A model describes a pattern in data. You do not need to write R to use these forms, but you do need to choose a method that fits your question. Start with an example whose question is close to yours.

A **response** is the result you want to explain or predict. **Predictors** are the columns you use to explain it. A **coefficient** describes part of the fitted relationship. A **fitted value** is the model's estimate for a row it used.

Run the input first. Check missing values and column types before selecting a model. After fitting, read warnings and the number of rows used. Use summaries and charts to check the data as well as the model output.

13.1 One-Way ANOVA

Use **One-Way ANOVA** to compare the means of a numeric response across groups.

Choose the response and one grouping column, then run the task. Check the group summaries before reading the test table. The p-value helps assess evidence that the group means are not all equal. It does not say which pairs differ or how large those differences are.

The method assumes a suitable study design and data pattern. Repeated measurements on the same person, for example, need more care than unrelated observations. **Plant growth - compare experimental groups** uses 30 plant weights in three groups. Open it to see the response, group choice, summary, and report together before trying your own comparison.

13.2 t Test

Use **t Test** to compare a mean with a chosen value or compare two groups, depending on the mode you select.

Choose the input and numeric response. For a two-group test, choose the group column and check that it contains the intended two groups. Leaving the group blank gives a one-sample test against zero. The form does not offer a different comparison value, so use that choice only when zero answers your question.

Check the paired setting carefully. For paired data, choose the second measurement column; each row must contain a matched pair, such as before and after values from the same person. Two unrelated groups are not paired. Run the task and read the mean difference and interval alongside the p-value. **Plant growth - compare experimental groups** includes a two-group example using a prepared subset.

13.3 Linear Models

Use **Linear Models** to explain a numeric response using one or more predictors, including groups.

Choose the response, then the predictors or model terms the form requests. A group column should be stored as a category when its values name groups. Run the task and check the model terms and number of observations.

Read the coefficient and fit tables together. A coefficient for a group is usually a comparison with a reference group; it is not that group's raw mean in every model. The **Plant growth** example shows a model beside its group comparisons. Keep the model description in your report so a reader knows which predictors you included. A completed fit alone does not show that the model is useful.

13.4 Linear Regression

Use **Linear Regression** to model a numeric response from selected predictors.

Choose the response and predictors, then enable the result tables you need. Run the task. Look at the coefficients, overall fit, fitted values, and residuals where offered. A residual is the observed value minus the fitted value.

The **Fuel economy model** example uses 32 cars and predicts fuel economy from weight and horsepower. In that dataset, weight is in thousands of pounds. Check those units before explaining a coefficient. A fit to these observations is not proof that changing one car feature will cause the predicted change. Try removing one predictor in a saved copy and compare both the model and its warnings.

13.5 Logistic Regression

Use **Logistic Regression** for a response with two possible outcomes, such as 0 and 1.

Choose the response and predictors. Check which outcome is coded as 1 before fitting. Run the task and inspect the fitted probabilities as well as coefficients. Probabilities range from zero to one. The coefficients use a different scale, so they are not direct percentage-point changes.

Cars - binary outcome model treats 1 as manual transmission and 0 as automatic. It is a useful way to check what a probability refers to. Read warnings about convergence or separation. These can mean the model cannot estimate the relationship reliably. A green task status does not make extreme estimates safe to use without checking them.

13.6 Generalized Linear Models

Use **Generalized Linear Models** when the response needs a model family other than ordinary numeric regression.

Choose the response, predictors, and **GLM family**. The family describes the kind of outcome. The form uses that family's standard link, which controls how predictors connect to the expected outcome. You do not choose a separate link here. Use a worked example or an analysis plan to guide the family choice.

Loom breaks - a count model uses a Poisson model for 54 counts of loom breaks. Counts are whole numbers of events, not percentages or rates. If exposure time differs between rows, the simple count example may not suit your question. Run the task and read warnings and fit information. A successful fit does not by itself show that the chosen family matches the data.

13.7 Nonlinear Regression

Use **Nonlinear Regression** to fit one of the supported curve forms to numeric measurements.

Choose the response, the predictor, and **Curve preset**. The task uses the first selected predictor and prepares its own starting values. Some curves require positive values, so inspect zeros and negative entries before running. Run the task, then compare the curve with the actual points.

Puromycin - fit a nonlinear curve uses positive concentration and reaction-rate values from a prepared subset. It shows a power curve as a teaching example. A different preset changes the model; it is not simply a repair button for a failed fit. If fitting fails, check valid input values and the selected form first. Be cautious about predictions beyond the observed range.

13.8 Correlations

Use **Correlations** to measure how pairs of numeric columns vary together.

Choose the numeric columns and run the task. It calculates Pearson correlations, which describe straight-line relationships. It uses available pairs when values are missing. Compare the table with scatter plots. Values near 1 or -1 show a strong straight-line relationship. Values near zero do not rule out a curved pattern.

A correlation does not show that one value causes another. Groups and unusual points can also change the result. **Flowers - multivariate structure** compares the four Iris measurements. Inspect the species groups as well as the combined result. Check the number of usable observations when missing values mean different pairs use different rows.

13.9 Principal Components

Use **Principal Components** to combine several numeric columns into fewer summary directions. The task is often called PCA.

Choose only the numeric measurements you want to combine. The task scales them before fitting, so you do not need to select a scaling option. Run the task. Check the variance explained, which shows how much of the data's spread each component represents. Loadings show how the original columns contribute.

Flowers - multivariate structure uses the four flower measurements and keeps species labels out of that input set. It uses scaling inside PCA. A component is a calculated direction, not a new measurement with a simple original unit. Use its scores to explore patterns, then return to the original columns to explain them.

13.10 Cluster Analysis

Use **Cluster Analysis** to group rows with similar numeric measurements.

Choose the measurement columns, the method, and the requested number of groups where needed. Set a seed for a repeatable random start. Check whether you need to standardize the inputs first: this task does not automatically put all columns on a common scale.

Run the task and inspect group sizes and assignments. Very small groups or results driven by one large-scale column need attention. **Flowers - multivariate structure** creates three clusters. Three requested clusters do not prove that the data contain three natural groups, and clusters need not match the known species. Compare the result with the source measurements and charts before giving the groups names.

14 Check patterns and work with time

These tasks answer more specific questions. Read the matching example first, and keep the meaning and order of your data clear.

14.1 CDF Plots

Use **CDF Plots** to see what share of observations is at or below a value. CDF stands for cumulative distribution function.

Choose a numeric measurement and run the task. Pick a value on the horizontal axis and read the share on the vertical axis. For example, a share of 0.75 means about 75% of the observations are at or below that value.

This view can help you compare thresholds without choosing histogram bins. Check the units and missing-value count. It describes the data you supplied, not a guarantee about future measurements. **Nile flow - distribution and stability** includes this plot alongside a probability plot and control chart, so you can compare different views of the same 100 annual observations.

14.2 Probability Plots

Use **Probability Plots** to compare the pattern of a numeric column with a reference distribution, such as a normal distribution.

Choose the measurement and available reference settings, then run the task. Points that follow the reference line suggest a similar pattern. Curves and distant points suggest differences, especially in the tails where extreme values lie.

The plot is a check, not a pass-or-fail certificate for every later analysis. Read it with a histogram and the study context. Do not remove observations just to make the points straighter. **Nile flow - distribution and stability** gives you a worked normal probability plot. Use it to learn how the line and points relate before interpreting your own data.

14.3 Control Charts

Use **Control Charts** to inspect measurements in their recorded order and flag unusual patterns.

Choose the measurement and check the row order. Time order matters here. Review the selected chart and rule settings, then run the task. Inspect any flagged points or runs.

The app estimates limits from the data. Those limits are different from an outside target or an allowed tolerance set by a customer. A point inside a limit is not proof that it meets a product requirement. **Nile flow - distribution and stability** uses an individuals chart for practice. River observations are a teaching dataset, not a controlled factory process. Read the example's notes before borrowing its settings for another purpose.

14.4 ARIMA / Basic Forecasting

Use **ARIMA / Basic Forecasting** to estimate future values from an ordered numeric series.

Choose the value column and set the forecast horizon, the number of future steps. Make sure each step means the same interval, such as one month. Sort and check the series before fitting. Run the task and inspect the fitted model summary, forecasts, and intervals.

Air passengers - forecast a monthly series starts with 144 monthly values and forecasts 12 more steps. The task does not automatically turn every dated table into a tuned seasonal model. Check the settings and model used. The intervals show model uncertainty; they do not promise that future events will stay within them. Keep the time labels clear when reporting the result.

15 Time until an event

Survival tasks use a time column and an event indicator. An event might be a failure, a repair, or another outcome. A censored observation means the event had not been seen by the end of that observation's follow-up.

For these tasks, check that **1 means the event occurred** and **0 means it was not observed during follow-up**. Other coding systems must be converted first. The example does this for you and explains the change in its preparation task.

15.1 Life Tables

Use **Life Tables** to estimate how the chance of remaining event-free changes over time.

Choose the time column and event column. Add a grouping column if you want separate curves, then run the task. Check the number of usable rows and events. Confirm that time uses one unit and does not contain unexplained negative values.

Lung cancer - survival analysis keeps the 228-row original data and prepares a separate input for analysis. Its original event code of 2 is converted to 1. Open the preparation task before reading the curve. The example teaches app controls and coding checks. It is not a guide for making care decisions. Use suitable analysis advice before applying these methods to important decisions.

15.2 Proportional Hazards

Use **Proportional Hazards** to model the relationship between predictors and the rate at which an event occurs over time. This is often called a Cox model.

Choose time, event, and predictor columns. Check event coding and missing values, then run the task. Read the coefficients, intervals, warnings, and number of included observations.

The **Lung cancer - survival analysis** example shows the prepared input and an illustrative model. A hazard ratio compares event rates in the fitted model. It is not an absolute probability that a person will remain event-free until a chosen date. The method also has assumptions about how relationships behave over time. Use the example to learn the app's settings, and get method advice when those assumptions affect your work.

16 Run steps conditionally or repeatedly

Most first projects can use a straight sequence of tasks. Add a Branch when a decision should choose a path. Add a Loop when the same steps should repeat. Try the supplied example before building a large version.

16.1 Branch

Use **Branch** to run tasks only when a condition is met. For example, you could make a summary only when a table has enough rows.

1. Add Branch and choose a condition. A row-count condition is a simple place to start.
2. Give the matching path a clear label. Add an Else path if you need an alternative when no ordinary path matches.
3. Open the task that should follow that decision. In **Run after branch/loop path**, select the intended path.
4. Run the flow and inspect the decision and skipped tasks in the log.

Choosing an input table alone does not attach a task to a branch path. That separate path choice is needed. **Flowers - conditional quality gate** checks for at least 100 flowers. Change the threshold in a saved copy and run again to see which path is taken.

16.2 Loop

Use **Loop** to repeat tasks for several values, files, tables, or groups.

Start with **Flowers - repeat by species**. Its group loop runs one summary for each species. Follow the connection from the Loop path to the repeated Script. Inspect the three saved summaries and the iteration entries in the log.

To build your own, choose the loop source, enter or select its items, and preview the planned iterations. An iteration is one trip through the repeated steps. Attach those steps through **Run after branch/loop path**. Choose output naming that keeps each result separate.

For exported files, give each iteration a different filename. A supported `{index}` placeholder can add the iteration number. If every iteration writes the same file with Overwrite on, only the last file may remain.

16.3 Choose a loop source

Source	What you provide
Parameter list	Values to use one at a time
Files	File paths or matching file patterns
Datasets	Available tables to process
Groups	A source table and grouping column
Date ranges	Dates or start/end ranges requested by the form

Source	What you provide
Database query results	A prepared result table whose rows supply the items

Check the preview before running. A missing file or a mistaken group can otherwise repeat the wrong work many times. If the repeated step uses code, begin with the working example and keep the loop variable name consistent with that code.

16.4 Start small

Try two or three iterations first. Check output names, file paths, row counts, and logs. Once those are right, expand the loop.

Place a final Report or Export Log outside the repeated path if it should run once at the end. Attach it inside the path only when you want one per iteration. A stopped or failed loop does not remove files that earlier iterations already wrote.

17 Keep a useful log

A log helps answer three questions: What ran? What changed? What went wrong? The app records task events, and you can add messages that explain your own checks.

17.1 Choose how much to record

Open a task's **Logging** section. Keep logging enabled, then choose a minimum level.

Level	Use it for
DEBUG	Extra details while finding a problem
INFO	Normal progress and results
WARN	Something to check, even if the task continues
ERROR	A failure or serious problem

INFO is a useful starting point. A minimum of WARN keeps warnings and errors but leaves out normal INFO messages. This choice affects which new messages are kept; raising the detail later cannot recover messages that were never recorded.

17.2 Customize a task's message

1. Open Logging and find the rule for the event you want, such as task completion.
2. Leave the rule enabled. Choose a level that fits the message.
3. Edit the message. Use the field picker and insertion buttons to add values, such as task name or output row count.
4. Open the logging preview and check the wording.
5. Run the task once and read the actual log entry.

For a table-cleaning task, you could use:

```
{task_name}: {rows_in} rows in; {rows_out} rows out.
```

The words in braces are placeholders. The app fills them with values from the run. Use the picker so you do not need to memorize their spelling. An output count may not exist at task start, so use it in a completion message.

Keep the original-message field in warning and error rules. That way, your friendly wording does not hide the reason something failed. If a placeholder stays visible, check the field and event in the preview.

17.3 Add a message inside a Script

This is optional. A Script can add a plain message with the supplied `log_event()` helper:

```
log_event("Checked the source data before making the report", "INFO")
```

Use messages to explain decisions, such as a filter or missing-value choice. Avoid printing private row-level information just to show that a step ran. The **Cars - a report with an execution audit** example includes a custom warning you can inspect.

A warning message alone does not stop a Script. If you need a quality condition to choose a different path, use a Branch. If you need code to fail on a bad condition, ask the code's author to make that check explicit.

17.4 Export Log

Use **Export Log** to write selected task logs to a file as part of your flow.

1. Add Export Log from Output.
2. Select the tasks to include and wait for. Include Report if you want its completion recorded.
3. Choose TXT, CSV, or JSON and the minimum level.
4. Choose a full destination path in a folder you can write to. Match the file ending to the format.
5. Set Overwrite only if reruns should replace that file. Run the task and open the file to check it.

TXT is easiest to read as plain text. CSV opens in a spreadsheet with separate fields. JSON is useful when another program will read the records. Export Log writes a file rather than making a new data table.

If the flow stopped at an earlier error, run Export Log by itself afterwards when possible. It can help you collect the error messages. Its file cannot include the exporter's own later completion message.

17.5 Put the log in a report

In **Report > Appearance**, turn on **Include pipeline log in the report**. Choose INFO to include normal progress, or WARN for warnings and errors only. Run the report and look for **Pipeline execution log**.

The report takes a snapshot of other tasks' available logs before it is made. It cannot include its own completion at that point. Add Export Log after Report if you need that event in a separate file. You can also make a report containing only a log by enabling the log option without selecting ordinary report items.

17.6 Read dates as well as messages

Tasks keep up to 1,000 recent entries each, across runs. A saved project keeps those available entries. Running a flow again does not erase its older messages.

Use the timestamps and run identifiers to compare runs. Old warnings may still appear after you fix a problem. Long or very chatty runs can push old entries out of the retained log, so export important logs in time.

17.7 Use startup logs for startup problems

The desktop **Session** menu offers **Open startup log**, **Open logs folder**, and **Save diagnostic log**. These help when the window cannot connect or the R server stops. Task logs help with the analysis itself.

Before sharing any log, check it for private data, passwords, and file paths you do not want to publish. Send the relevant file and a short description of what you were doing.

18 Share reports and files

Choose what your reader needs. Send a report for reading, a CSV for working with data, or a project for editing the flow. You can send more than one when each has a clear purpose.

18.1 Report

Use **Report** to bring tables, charts, and notes into an HTML or Word document.

1. Add Report from Output.
2. Under **Content**, select the results to include. Put them in a useful order.
3. Add clear headings and short notes. Explain the data source, units, and important choices.
4. Under **Appearance**, choose HTML or Word and enter the report title.
5. Turn on **Include pipeline log in the report** if you want an execution-log section.
6. Run the task, then open or download the report.

Check the document itself before sharing it. A missing result can lead to a warning and an incomplete section. Run its source task, then regenerate the report. **Iris exploration** shows tables and a chart together. **Cars - a report with an execution audit** focuses on logging.

18.2 Choose HTML or Word

HTML opens in a web browser. It is a good first choice and can include interactive tables where supported. Download it and check it outside the running app.

Word produces a .docx file that you can edit in a compatible word processor. Use static charts and tables; a Word page cannot keep browser hover controls. If your chart is missing, enable its static image output, run the chart, and select that output in the report.

The Windows Setup includes the normal report tools. An R-only installation needs extra reporting packages, and Word also needs Pandoc. See Optional: install with R if that is how you run the app.

18.3 Pick a content style

Start with **Auto**. It chooses a display based on the actual result. Use a table style for a table and a figure style for a static chart. Print style shows an object's text when that is what you need.

Give a reader enough context to understand a result. For example: **Mean petal length by species, 150 flowers, measurements in centimetres** is more useful than **Output 2**. If you change the data or chart, run Report again; a previously saved document will not update on its own.

18.4 Export

Use **Export** to write a table, chart, or supported object to an outside file.

Choose the source result first. Then select a compatible format, a destination folder, and a filename. For tables, check any selected-column settings. For images, check dimensions and resolution. Run the task and open the file from the chosen folder.

Start with CSV for data and PNG for a still chart. The source must be the right kind of result: exporting chart data as CSV is different from exporting the chart's static image as PNG.

Check Overwrite before rerunning. Append is available for CSV and TSV when supported; it adds rows to an existing file. It is not a general way to add a sheet to Excel. An Export task writes a file and does not create a new table for later tasks. The original result remains the source for those tasks.

18.5 Choose an export format

What you need	A useful choice	What to check
Data for many other apps	CSV or TSV	Dates, text IDs, and missing values after opening
A spreadsheet	XLSX	Column types and the receiving spreadsheet's limits
A still chart	PNG	Readable labels, dimensions, and resolution
A chart that can scale	PDF or SVG, when offered	That the receiving app can open it
Typed data for compatible tools	Parquet or Feather	Support in the receiving tool
Structured data for a program	JSON	The format expected by that program
An R result for another R user	RDS or RData	The recipient's R and package support

You do not need to learn every format. Choose the one your reader or next tool can use. If leading zeros disappear in a spreadsheet, import the ID column as text in that spreadsheet.

18.6 Check a finished delivery

Save the project. Open the exported report and data files outside the app. Check titles, units, row counts, charts, and the log's dates. Make sure you are sending the latest files from the folder you actually chose.

If you need a record that the report finished, send a final Export Log made after Report. Its events are later than the snapshot inside the report.

19 Optional: try the R Console

You can use the app without the console. It is there for quick questions and small trials when you want more control.

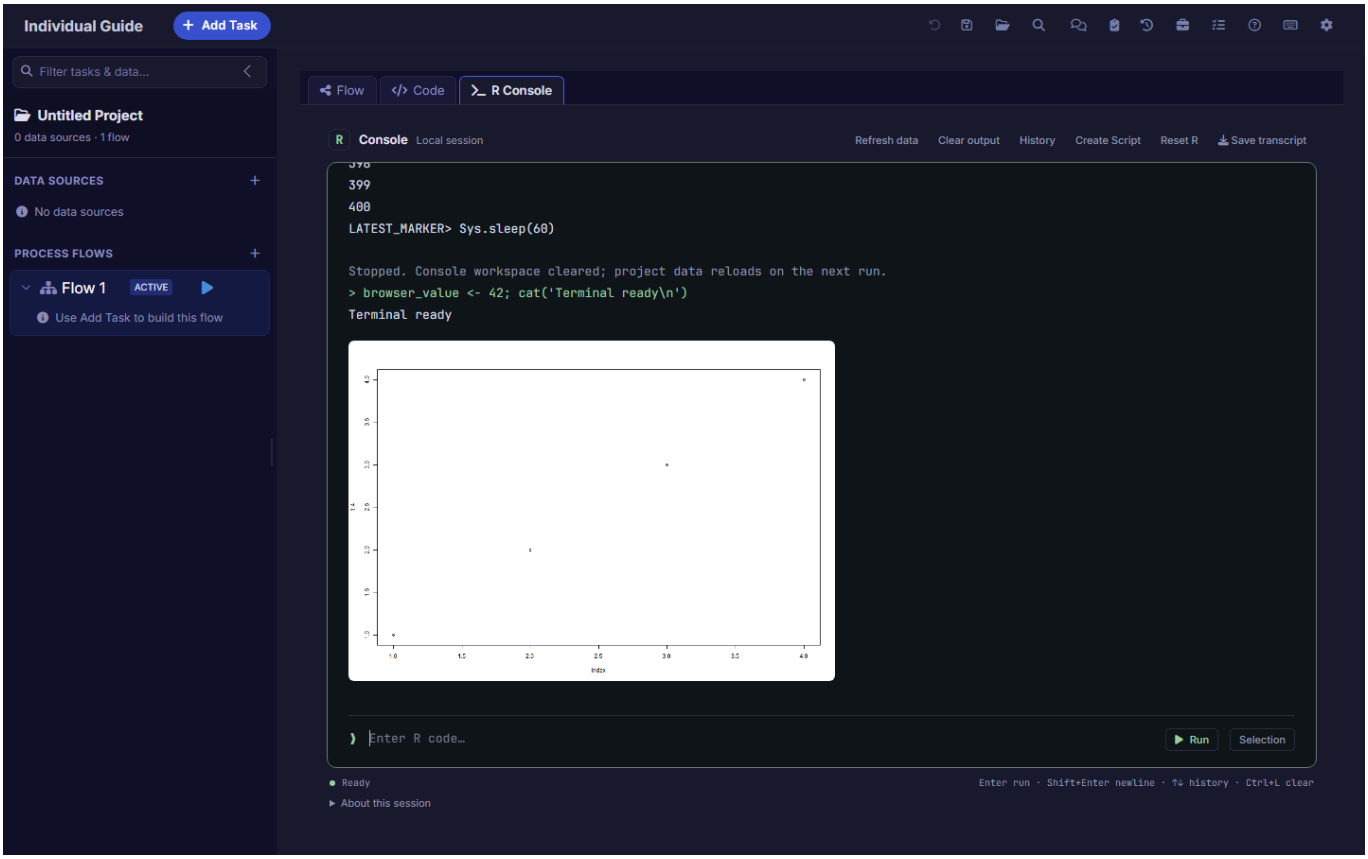
R is the calculation language behind the app. A **command** is an instruction you ask R to carry out. The console shows the command and its result. You can start with a harmless calculation:

```
2 + 3
```

The result is 5. You do not need to install anything to try this inside the desktop app.

19.1 Open the console and run a command

Choose the **R Console** tab. In the desktop app, you can also use **View > R console**. Type in the command area, then press Enter for a one-line command. Use Shift+Enter to add a line and Ctrl+Enter to run a block.



The R Console has a command area, results, and controls for working with project data.

If you want to inspect a project table, use the console's project-data controls to add it. Use its exact table name in commands. These are working copies; changing a console value does not automatically change the project's table or flow.

19.2 Read a command before running it

Commands can calculate, change values, read files, or write files. Use code you understand or trust. For a simple practice with R's built-in flower data, try:

```
mean(datasets::iris$Sepal.Length)
```

The result is about 5.8433. Here, `mean` asks for an average. The rest identifies the sepal-length column in the built-in flower data. You do not need to memorize that punctuation to use the task forms.

19.3 Find and reuse a command

Use the history controls to find earlier commands. Up and Down move through history when the cursor is at the command boundary; Alt+Up and Alt+Down also help. Ctrl+R searches command history.

Tab or Ctrl+Space offers completions when available. A completion helps type a name; it does not check that the finished command answers your question. Read errors and edit the command before trying again.

19.4 Clear, Stop, and Reset

Action	When to use it	What to expect
Clear or Ctrl+L	You want a cleaner display	Clears visible output; working values remain
Stop	A command is taking too long	Stops console work and clears its working values; the transcript and history remain
Reset	You want a fresh console	Clears its working values, transcript, history, draft, and plots
Restart R server	The app's calculation service has stopped	Restarts the app session; save first if possible

Stopping console work can mean you need to load your project-table copies again. These actions do not turn a trial command into a saved task.

19.5 Keep useful code in a task

If a command should be part of the analysis every time, use **Create Script** or copy it into a Script task. Declare the input tables and named outputs there. Run the task and check its results.

Use task logging for the saved analysis. The console transcript records your trials, which may include commands you later changed or abandoned.

20 Optional: export the flow as R code

The **Code** tab can export the active flow as a complete R script or a code bundle. This is useful when someone needs to rerun the analysis without opening Individual Guide.

A script is a text file of R commands. A bundle includes the script and supporting files with instructions. Choose a complete-flow export for a handoff; an individual task's code may depend on earlier steps.

You can skip code export for everyday app use. A saved project is usually the easiest way to return to your work.

20.1 Share code that someone can run

Before exporting, run the flow and check its results. Choose the flow you mean to share, then export its full script or bundle from Code. Include needed input files and tell the recipient which outputs to expect.

The recipient needs R, the packages requested by the script, and access to its inputs. The complete generated pipeline does not need the Individual Guide package itself. Database access and file locations still need to be set up on the receiving computer.

Open the bundle's README and follow its instructions in a separate working folder. Check output destinations before running. Running a saved script can replace files just as running Export inside the app can.

21 Explain and review your work

A useful project tells another person what you did and why. A few clear notes can save much more time than a long trail of unexplained task names.

21.1 Add notes at decision points

Use task notes or project comments to explain choices. Record why you removed rows, filled missing values, chose a model, or set a threshold. Write for someone who sees the project for the first time.

For example: **We kept orders from January through March and excluded cancelled orders.** That tells a reader more than **Filter updated**.

21.2 Check before sharing

Open the Review tools to inspect the project. Use History to see changes and Run records to inspect previous executions. A run record can help compare the settings and inputs used for different results.

Check the current results too. Old success messages do not prove that a changed task has run again. Update stale results and the report, then save.

21.3 Find something quickly

Use project search for task names and other saved information. Open the Task list when the diagram is crowded. Clear names help both tools work better.

Undo can restore supported project changes, but it does not rewind every action. In particular, it cannot bring back the old contents of an outside file that Export overwrote. Save a separate copy before a large change.

21.4 A simple review checklist

- Can you name the data source and the date of the copy you used?
- Do row counts and missing values make sense after cleaning?
- Do charts show the correct columns, labels, and units?
- Have you checked model warnings and the number of rows used?
- Does the latest report match the latest results?
- Are logs and exported files in the folder you expect?
- Have you saved the project and checked the copy you will share?

22 Make the app comfortable to use

Open **Settings** from the desktop File menu or the app's project tools. Start with the defaults, then change one setting at a time.

22.1 Display and navigation

Adjust theme and display choices if they help you read the workspace. Use desktop zoom controls if text is too small. For a large flow, combine Fit view with the Task list rather than trying to read every small shape at once.

Keyboard navigation can help when a diagram is difficult to use. Tab moves focus through controls. The shortcut guide lists common desktop keys.

22.2 Saving and confirmation choices

Check recovery and saved-version settings before relying on them. The usual saved-version limit is 20; you can change it in settings. Keep your own backups for important projects.

Leave useful confirmations on while learning. Read any warning about unsaved work before opening another project or example. Saving takes a moment and is easier than rebuilding changes later.

22.3 Trust and private information

Opening a project can make its saved tasks and code available to run. Accept projects from sources you trust. Review input paths, export destinations, and Script tasks before running someone else's flow.

Treat a project as a file that may contain private information. A report or log can contain it too. Check the actual files before putting them in a public folder or attaching them to an issue.

23 Keep work running smoothly

Try a small version of a flow before using a large dataset. Check its settings, then run the full version once you know what it will do.

23.1 When editing feels slow

Wait for an active run to finish, or use Stop if you do not need it. Close task panels you are not using and use the Task list to move around a large project. A smaller chart or sample can make exploration easier, but keep full data for calculations that need it.

If the app remains slow, save, close it, and open it again from its shortcut. Record which task and action cause the delay if you need help. A report such as **Typing the output name is slow in a 40-task project** is easier to investigate than **The app is slow**.

24 Troubleshooting

Start with the symptom below. Keep the exact error text when asking for help. It often points to a missing file, column, or package.

24.1 The app seems not to open

Use the installed Start menu or desktop shortcut. Wait for the startup stages, especially on the first launch. If you opened Setup, finish installation and then use the app shortcut.

If startup shows a failure, open its startup log. Save the error text. Check that the installer came from the public release page and that it matches your 64-bit Windows computer.

24.2 The app disconnects or the R server stops

In the desktop app, choose **Session > Restart R server**. Save first if the workspace still responds. Restarting stops active work and can lose unsaved changes. After reconnecting, reopen your saved project or use the offered recovery copy, then check results before running again.

When running through R in a web browser, go to the R window that launched the app. Stop that session if needed and run the launch command again. See Optional: install with R. The browser version has no independent desktop Session menu.

24.3 A button or menu item is disabled

Finish any open dialog and check whether a flow is running. Open or create a project if you are still on the welcome screen. If the app is disconnected, reconnect or restart the R server first.

24.4 A file will not import

Check that the file still exists and that you can open its folder. For text files, check the separator and header. For Excel, check the selected sheet and range. Confirm the file type matches the import choice.

If the error names a missing package and you run through R, follow the public package instructions. If you use Setup, keep the full error and app version when reporting the issue.

24.5 A table or column is missing from a choice list

Run the task that creates the table. Check its output name and Results. Then check whether a cleaning step renamed or removed the column. A numeric-only list may hide a column stored as text; inspect its type before converting it.

24.6 A result is old, empty, or different from the preview

Check the selected input and filters. Use Run, rather than only Preview, to keep the result. Run later tasks again when their inputs change. Check branch decisions for skipped work and logs for failed earlier tasks.

Some views show only the first part of a large table. Read the row-count and limit information before assuming the full data are missing.

24.7 A model fails or looks strange

Read its first warning or error. Check response and predictor types, missing values, group counts, and the number of usable rows. Start with a matching example to check the setup. A model may need method advice even when all app settings are valid.

24.8 A report is missing a chart

Run the chart task and enable its static image output. Select that image in Report, then run Report again. Open the downloaded file to check the result.

If Word generation says Pandoc is missing, use the reporting setup in the optional R section. The Windows Setup already includes Pandoc. HTML is a useful first format to try.

24.9 An exported file is missing or will not update

Open its Export or Export Log task. Check the full destination, folder permissions, filename, and format. Run the task again. If the file already exists, decide whether to allow Overwrite or use a new name. Close the file in another program if it is locked.

24.10 A log seems empty

Check that task logging is enabled and that its rules are enabled. INFO messages will not appear with a minimum of WARN. Run a selected task again and check its own log before checking an export or report appendix.

The report's log section contains earlier recorded events, not its own later completion. Put Export Log after Report when you need that entry.

24.11 Ask for help

Use the public issue tracker for app problems. A GitHub account is needed to create an issue. Include the app version, how you opened it, the last action, the exact error, and what you expected.

For startup or connection problems, use **Session > Save diagnostic log** in the desktop app. For an analysis problem, export the relevant task log. Check attachments for private data and passwords before posting them. A small example using public data is often the easiest way to explain a problem.

25 Optional: install with R

Skip this chapter if you use the Windows Setup. These steps are for people who want to launch the app from R instead. You do not need to learn general R programming first.

R is the software that performs calculations. An R **package** is an add-on. IndividualGuide is the app's package name. You install it once, then use a short command to open the app on later days.

25.1 Step 1: install R from its public website

Go to the R download site, CRAN. Choose your operating system. On Windows, choose **Download R for Windows**, then **base**, then the current R installer. Follow the installer's instructions.

Open **R** from the Start menu. You should see a console with a `>` prompt. That is where you paste the commands below. Do not type the `>` itself, and do not paste these installation commands into an app Script task.

On macOS, use the CRAN macOS download and open the R application. Linux users should follow CRAN's instructions for their Linux version; extra system software may be needed for some packages.

25.2 Step 2: install the app from GitHub

You need internet access for installation. In the R console, paste these commands and press Enter. Installation can take several minutes and print many progress lines. Keep R open and wait for a new `>` prompt before moving on.

```
options(repos = c(CRAN = "https://cloud.r-project.org"))
install.packages("remotes")
remotes::install_github(
  "woodtho/IndividualGuide",
  ref = "v0.4.0",
  dependencies = NA,
  upgrade = "never"
)
```

This gets the app from its public GitHub release tag and its required add-ons from the public CRAN package service. It does not require you to download the source repository or find a local build folder. You normally do not need a GitHub account for this public download.

If R asks to create a personal package library, choose Yes. This means a folder for your installed add-ons. If Windows asks whether to compile packages from source when a ready-made version is available, choose No to use the ready-made version. If installation ends in an error, read it before trying to launch the app.

25.3 Step 3: add the features you need

The basic app installation includes its required packages. These extra public packages support reports and some analysis tasks. Install them once if you want to use those features or explore all the examples:

```
install.packages(
  c("rmarkdown", "knitr", "survival", "forecast"),
  repos = "https://cloud.r-project.org"
)
```

For Word reports, also install Pandoc using its official installer. Close and reopen R after installing it. You can check that R can find it with:

```
rmarkdown::pandoc_available()
```

`TRUE` means it was found. HTML reports have a simpler fallback, so you can start with HTML while sorting out Word support.

Only add database support if you need it. Install `RSQLite` for SQLite, `duckdb` for DuckDB, or `odbc` for an ODBC connection using `install.packages()` and the same CRAN address. ODBC may also need a driver from the database provider. Ask your database support contact which one to use.

25.4 Step 4: open the app

In the R console, run:

```
IndividualGuide::run_app()
```

The app opens in a web browser. Keep the R window open while using it. That R session is running the app, so it is normal for its console to be busy.

Use the app's own open and save buttons. The desktop File and Session menus belong to the Windows Setup version and are not present here.

When you finish, save the project first. Close the app's browser tab, then stop the app in R with the Stop control or Esc as appropriate for your R window. Quit R when you no longer need it.

25.5 Open it next time

Open R and run only `IndividualGuide::run_app()`. You do not need to reinstall packages each day. If the browser does not open automatically, use the local address printed in the R console.

To update later, save and close the app, start a fresh R session, and follow the instructions for the release you want. The tag `v0.4.0` deliberately selects the version used in this guide.

25.6 If package installation fails

Check your internet connection and copy the first useful error. A message about a missing system library or compiler needs the setup for that package and operating system; repeating the same command will not fix it. On Windows, the public Setup is the easier route if you do not need an R-managed installation.

The commands above follow the official remotes installation reference. Keep using the project’s public release notes for version-specific instructions.

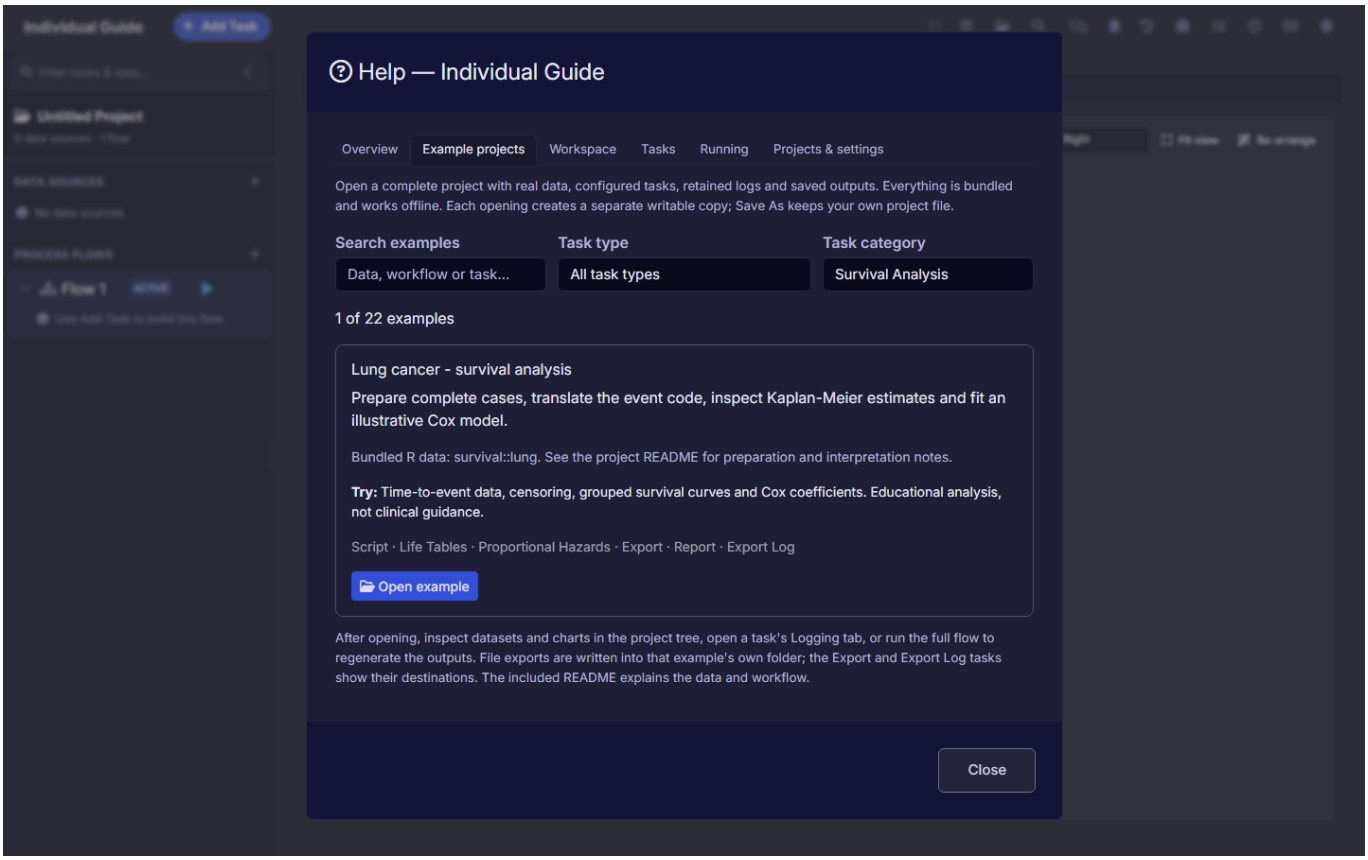
26 Choose an example project

All 22 projects below come with data, configured tasks, saved results, logging, and output files. They use public sample data supplied with R or its packages. You do not need a separate data download to explore them.

26.1 Find an example by task

1. Open **Help > Example projects** in the app.
2. Search for a title or topic. Use the **task type** filter to find a specific task, such as Rename Columns or Cluster Analysis.
3. Use the **category** filter to narrow the list to a group, such as Graph or Survival Analysis.
4. Choose **Open example**, then use **Save As** to keep your own copy.

If nothing appears, clear the search and filters, then try just one filter. You can also use the Task finder at the end of this guide to see which projects contain a task.



Example projects filtered to the Survival Analysis category in Help.

26.2 A simple learning route

Start with **Iris exploration**. Next try **Cars - prepare an analysis table** for cleaning, **Flowers - categorical chart gallery** for charts, and **Cars - a report with an execution audit** for reporting and logs. Choose models and control-flow examples when you need them.

26.3 Before you rerun an example

Read its notes and inspect its saved results first. Check Export and Export Log destinations before running the flow. Each opened example gets a separate working copy, but you should still choose folders and Overwrite settings that suit your own work.

These projects teach app controls. Their model and cleaning choices are examples, not a claim that those choices fit every dataset.

26.4 Iris exploration

Explore measurements from 150 flowers: species summaries, a scatter plot, an HTML report and exported results.

Data: 150 flowers, with 50 in each of three species.

Tasks to find: Script, Summary Statistics, Summary Tables, Scatter Plot, Export, Report, Export Log.

Check the result: Open the species summary. Mean petal lengths are about 1.462, 4.260, and 5.552 centimetres.

Try this next: Change the scatter plot's Y column to Petal.Width. Run the chart and report again.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.5 Air quality - clean and compare

Clean 153 daily New York air-quality observations, inspect missing values and compare ozone measurements with temperature.

Data: 153 daily New York air-quality observations, including missing measurements.

Tasks to find: Script, Summary Statistics, Replace Missing Values, Scatter Plot, Export, Report, Export Log.

Check the result: Compare the raw table, the missing-value audit, and the cleaned result. The example uses mean replacement as a teaching choice.

Try this next: Read the replacement task's settings. In a copy, try another available method and compare the summary. Keep the raw data.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.6 Fuel economy model

Use 32 Motor Trend cars to model fuel economy from weight and horsepower, inspect coefficients, fitted values and a model report.

Data: 32 cars, with fuel economy, weight, horsepower, and other measurements.

Tasks to find: Script, Summary Statistics, Linear Regression, Scatter Plot, Export, Report, Export Log.

Check the result: Open the model coefficients and fitted values. Weight is measured in thousands of pounds.

Try this next: Remove one predictor in a copy. Rerun the model and report and compare the results.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.7 Cars - prepare an analysis table

Turn car measurements into a labelled, deduplicated analysis table and inspect each intermediate result.

Data: 32 original cars plus one duplicate row added for practice.

Tasks to find: Script, Remove Duplicates, Keep / Drop Columns, Rename Columns, Derive Columns, Recode Values, Change Column Types, First / Last N Rows, Characterize Data, Export, Report, Export Log.

Check the result: The duplicate-removal step reduces 33 rows to 32. Inspect the renamed columns and transmission labels.

Try this next: Open each cleaning task in order. Compare the table before and after it, then change one label in a saved copy.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.8 US states - import, join and filter

Import a bundled CSV of state statistics, join a regional lookup, filter and sort the result.

Data: A bundled CSV with 50 US states, plus a state-to-region lookup.

Tasks to find: Script, Import Data, Query Builder, Filter and Sort, Export, Report, Export Log.

Check the result: The left join should retain the states and add region labels. The later filter keeps Income of at least 4500 and sorts it in descending order.

Try this next: Change the income threshold. Check which rows remain and whether the sort order still makes sense.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.9 Iris - reshape and compare

Pivot flower measurements to long form and back, compare the round trip, append species subsets and transpose a small table.

Data: The 150 Iris flowers with an added flower identifier.

Tasks to find: Script, Stack Columns, Split Columns, Transpose, Compare Data, Append Table, Export, Report, Export Log.

Check the result: Stacking four measurement columns produces 600 rows. The identifier helps put the measurements back into columns. Two appended species subsets contain 100 rows.

Try this next: Compare the long and wide tables. Inspect Compare Data before assuming the round trip preserved every detail.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.10 Cars - rank, scale and sample

Rank fuel economy, standardize predictors and draw a reproducible sample with an explicit seed.

Data: Measurements from 32 cars.

Tasks to find: Script, Rank, Standardize Data, Random Sample, Export, Report, Export Log.

Check the result: Inspect the rank and standardized columns before the ten-row sample. The sample uses seed 2026.

Try this next: Run the sample again with the same data, then change the seed. Compare row identities as well as the count.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.11 Tooth growth - descriptive analysis

Inspect column structure, list observations, count supplement groups, cross-tabulate dose and summarize tooth lengths.

Data: 60 tooth-growth observations, with supplement and dose groups.

Tasks to find: Script, Characterize Data, List Data, One-Way Frequencies, Summary Statistics, Summary Tables, Table Analysis, Distribution Analysis, Export, Report, Export Log.

Check the result: Compare the frequency counts, dose-by-supplement table, and numeric summaries. Dose is stored as a category for the grouped examples.

Try this next: Choose a different grouping in a copy. Check what the counts and percentages refer to before describing them.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.12 Flowers - categorical chart gallery

Compare species with bars, a pie chart and grouped box plots, then collect the figures into a report.

Data: 150 flowers in three equal-sized species groups.

Tasks to find: Script, Bar Chart, Pie Chart, Box Plot, Export, Report, Export Log.

Check the result: The species counts are 50 each. Compare bars, pie slices, and grouped box plots in the report.

Try this next: Change chart titles or category styles. Check that the report's static figures update after you rerun it.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.13 Flowers - numeric chart gallery

Explore numeric relationships with bubbles, density contours, histograms, scatter plots and a scatter matrix.

Data: The four numeric measurements from the 150 Iris flowers.

Tasks to find: Script, Bubble Plot, Contour Plot, Histogram, Scatter Plot, Scatter Plot Matrix, Export, Report, Export Log.

Check the result: Compare the histogram, bubble plot, contour plot, scatter plot, and matrix. Each view answers a different question.

Try this next: Change histogram bins or a selected measurement. Check axis labels and the chart's data limit.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.14 Nottingham temperatures - time charts

Chart twenty years of monthly temperatures and compare annual averages with their observed range.

Data: 240 monthly Nottingham temperature observations, with dates added for charting.

Tasks to find: Script, Line Plot, Bar-Line Chart, Export, Report, Export Log.

Check the result: Compare the monthly line with the annual average-and-range chart. The annual range is not a confidence interval.

Try this next: Inspect the date order. Change a title to include the temperature units and check both axes.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.15 Plant growth - compare experimental groups

Compare plant weights across control and treatment groups, with ANOVA, a two-group t test and a general linear model.

Data: 30 plant weights in one control and two treatment groups.

Tasks to find: Script, One-Way ANOVA, t Test, Linear Models, Export, Report, Export Log.

Check the result: The ANOVA uses all three groups. The t test uses a prepared two-group subset, so the tests do not ask exactly the same question.

Try this next: Inspect the subset before the t test. Read group counts and means before the test tables.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.16 Cars - binary outcome model

Model manual versus automatic transmission with a logistic regression and retain fitted probabilities and coefficients.

Data: 32 cars, with transmission coded as 0 for automatic and 1 for manual.

Tasks to find: Script, Logistic Regression, Export, Report, Export Log.

Check the result: The fitted probabilities describe the outcome coded 1. They use a different scale from the coefficients.

Try this next: Inspect the response coding, then try a different predictor in a copy. Read any fit warnings.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.17 Loom breaks - a count model

Fit a Poisson model to counts of loom breaks by wool and tension, and retain model diagnostics.

Data: 54 loom-break counts, with wool and tension groups.

Tasks to find: Script, Generalized Linear Models, Export, Report, Export Log.

Check the result: The model uses a Poisson family for counts. Read the fit information rather than treating completion as proof of a good model.

Try this next: Inspect the selected response and groups. Compare count summaries with the model results.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.18 Puromycin - fit a nonlinear curve

Fit a power curve to enzyme reaction rates in the treated sample and compare it with the observations.

Data: Puromycin enzyme measurements, filtered to the treated sample.

Tasks to find: Script, Nonlinear Regression, Filter and Sort, Export, Report, Export Log.

Check the result: The example fits a power curve to positive concentration and rate values. Read the preparation filter and compare the curve with the points.

Try this next: Inspect the selected curve and input range. Avoid explaining predictions far beyond the observed measurements.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.19 Flowers - multivariate structure

Explore correlations, principal components and three clusters using the four flower measurements.

Data: Four numeric measurements from 150 flowers; species is kept as a known label.

Tasks to find: Script, Correlations, Principal Components, Cluster Analysis, Export, Report, Export Log.

Check the result: Compare correlations, scaled principal components, and three clusters. Clustering does not automatically use the same scaling as PCA.

Try this next: Inspect the cluster sizes and assignments. Compare them with species without assuming they must match.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.20 Lung cancer - survival analysis

Prepare complete cases, translate the event code, inspect Kaplan-Meier estimates and fit an illustrative Cox model.

Data: 228 original lung-cancer observations, with a separate prepared analysis table.

Tasks to find: Script, Life Tables, Proportional Hazards, Export, Report, Export Log.

Check the result: The preparation converts the original status code 2 to event 1 and selects complete cases. Compare row counts before and after preparation.

Try this next: Read the time and event settings before the curves or model. Use this example to learn the controls, not to make care decisions.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.21 Air passengers - forecast a monthly series

Forecast twelve months of airline passengers with the app's ARIMA task, retaining point forecasts and intervals.

Data: 144 ordered monthly airline-passenger values.

Tasks to find: Script, ARIMA / Basic Forecasting, Export, Report, Export Log.

Check the result: The horizon is 12 future steps. Inspect the model summary and intervals. This is a basic forecasting example, not an automatically tuned seasonal analysis.

Try this next: Change the horizon in a copy. Check the number of forecast rows and explain what one step means.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.22 Nile flow - distribution and stability

Inspect annual river flow with an individuals chart, empirical CDF and normal probability plot.

Data: 100 annual Nile-flow observations in their original order.

Tasks to find: Script, Control Charts, CDF Plots, Probability Plots, Export, Report, Export Log.

Check the result: Read the control-chart flags beside the CDF and normal probability plot. The limits are estimated from these observations.

Try this next: Inspect unusual points and their years. Keep the time order and distinguish chart limits from outside requirements.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.23 Flowers - conditional quality gate

Use a row-count condition to activate a summary path and record the gate decision in the audit log.

Data: 150 flowers and a condition requiring at least 100 rows.

Tasks to find: Script, Branch, Summary Statistics, Export, Report, Export Log.

Check the result: The summary follows the chosen branch path. Check its Run after branch/loop path setting, not only its table input.

Try this next: Raise the threshold above 150 in a copy. Run again and inspect the decision and skipped work in the log.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.24 Flowers - repeat by species

Loop over the three species, compute a separate summary per iteration and keep iteration-aware logs.

Data: 150 flowers split into three species groups.

Tasks to find: Script, Loop, Export, Report, Export Log.

Check the result: Find species_summary_1, species_summary_2, and species_summary_3. Each summarizes a group of 50 flowers.

Try this next: Read the iteration entries in the log. Check that the final report and log export sit outside the repeated body.

Open its report and log exports, then save your project copy if you want to keep your changes.

26.25 Cars - a report with an execution audit

Create a results report with a pipeline-log appendix, a custom warning and separate text, CSV and JSON log files.

Data: 32 cars, a numeric summary, and a custom teaching warning.

Tasks to find: Script, Summary Statistics, Export, Report, Export Log.

Check the result: Open the report's Pipeline execution log and compare the TXT, CSV, and JSON log exports. The later export can include report completion.

Try this next: Set the report appendix to WARN and rerun it. Compare its shorter log with the INFO version and the full export.

Open its report and log exports, then save your project copy if you want to keep your changes.

27 Task finder

Find a task below, then follow its link for setup steps. The example links show up to three projects you can open from Help. Use the app's task filter to find all matches. Start by inspecting a completed example rather than rebuilding it from scratch.

Task	Where to find it in Add Task	Examples to open
Import Data	Data	US states - import, join and filter
Query Builder	Data	US states - import, join and filter
Script	General	Iris exploration; Air quality - clean and compare; Fuel economy model
Branch	Control Flow	Flowers - conditional quality gate
Loop	Control Flow	Flowers - repeat by species
Append Table	Data	Iris - reshape and compare
Compare Data	Data	Iris - reshape and compare
Filter and Sort	Data	US states - import, join and filter; Puromycin - fit a nonlinear curve
Stack Columns	Data	Iris - reshape and compare
Split Columns	Data	Iris - reshape and compare
Transpose	Data	Iris - reshape and compare
Rank	Data	Cars - rank, scale and sample
Standardize Data	Data	Cars - rank, scale and sample
Random Sample	Data	Cars - rank, scale and sample
Remove Duplicates	Data	Cars - prepare an analysis table
Keep / Drop Columns	Data	Cars - prepare an analysis table
Rename Columns	Data	Cars - prepare an analysis table
First / Last N Rows	Data	Cars - prepare an analysis table
Replace Missing Values	Data	Air quality - clean and compare
Derive Columns	Data	Cars - prepare an analysis table
Recode Values	Data	Cars - prepare an analysis table
Change Column Types	Data	Cars - prepare an analysis table
Characterize Data	Describe	Cars - prepare an analysis table; Tooth growth - descriptive analysis
Distribution Analysis	Describe	Tooth growth - descriptive analysis
List Data	Describe	Tooth growth - descriptive analysis
One-Way Frequencies	Describe	Tooth growth - descriptive analysis

Task	Where to find it in Add Task	Examples to open
Summary Statistics	Describe	Iris exploration; Air quality - clean and compare; Fuel economy model
Summary Tables	Describe	Iris exploration; Tooth growth - descriptive analysis
Table Analysis	Describe	Tooth growth - descriptive analysis
Bar Chart	Graph	Flowers - categorical chart gallery
Bar-Line Chart	Graph	Nottingham temperatures - time charts
Box Plot	Graph	Flowers - categorical chart gallery
Bubble Plot	Graph	Flowers - numeric chart gallery
Contour Plot	Graph	Flowers - numeric chart gallery
Histogram	Graph	Flowers - numeric chart gallery
Line Plot	Graph	Nottingham temperatures - time charts
Pie Chart	Graph	Flowers - categorical chart gallery
Scatter Plot	Graph	Iris exploration; Air quality - clean and compare; Fuel economy model
Scatter Plot Matrix	Graph	Flowers - numeric chart gallery
One-Way ANOVA	ANOVA	Plant growth - compare experimental groups
t Test	ANOVA	Plant growth - compare experimental groups
Linear Models	ANOVA	Plant growth - compare experimental groups
CDF Plots	Capability	Nile flow - distribution and stability
Probability Plots	Capability	Nile flow - distribution and stability
Control Charts	Control Charts	Nile flow - distribution and stability
Linear Regression	Regression	Fuel economy model
Logistic Regression	Regression	Cars - binary outcome model
Generalized Linear Models	Regression	Loom breaks - a count model
Nonlinear Regression	Regression	Puromycin - fit a nonlinear curve
Correlations	Multivariate	Flowers - multivariate structure
Principal Components	Multivariate	Flowers - multivariate structure
Cluster Analysis	Multivariate	Flowers - multivariate structure
Life Tables	Survival Analysis	Lung cancer - survival analysis
Proportional Hazards	Survival Analysis	Lung cancer - survival analysis
ARIMA / Basic Forecasting	Time Series	Air passengers - forecast a monthly series
Export Log	Output	Iris exploration; Air quality - clean and compare; Fuel economy model
Export	Output	Iris exploration; Air quality - clean and compare; Fuel economy model

Task	Where to find it in Add Task	Examples to open
Report	Output	Iris exploration; Air quality - clean and compare; Fuel economy model

28 Shortcuts and everyday terms

28.1 Useful Windows desktop shortcuts

Use the menus while learning. These keys can save time once the actions feel familiar. A text editor or open dialog may use a key for its own controls.

Keys	Action
Ctrl+N	New project
Ctrl+O	Open project
Ctrl+S	Save project
Ctrl+Shift+S	Save As
Ctrl+Shift+F	Search project
Ctrl+Alt+L	Open Task list
Ctrl+Alt+R	Run active flow, when focus is outside an editor
Ctrl+Shift+F5	Stop the flow run
Ctrl+Shift+R	Restart R server; save first if possible
F1	Open app help
Tab / Shift+Tab	Move focus through controls

For more keys, open **Help > Keyboard shortcuts**. If you run in a normal web browser, use the app buttons; the browser may use these same keys for its own actions.

28.2 A few more terms

Term	Plain meaning
Column type	The kind of value stored, such as text, number, date, or category
CSV	A text data file whose columns are separated by commas
Input	A table or result that a task needs
Missing value	An unknown entry; not automatically zero
Object	A saved result that is not an ordinary table, such as a model
Overwrite	Replace an existing file with a new one
Package	An add-on that gives R more features
Path	The address of a file or folder
Predictor	A column used to help explain or predict a response
Response	The outcome a model tries to explain or predict

Term	Plain meaning
Seed	A setting for repeating a random choice with the same inputs
Stale	A result that needs to be rerun after a change

28.3 Public downloads and help

- Individual Guide releases: Windows Setup, R package files, and downloadable handbook editions.
- Release 0.4.0: the version used by this guide.
- Report an app issue: include the version and exact error; check attachments for private information.
- Download R: needed only for the optional R installation route.
- Install Pandoc: for Word reporting when using your own R installation.

The example data come from R's datasets package and the survival package. The example notes describe prepared fields and teaching choices. Individual Guide is an open-source project by Thomas Wood.

Made for people using Individual Guide 0.4.0. Find downloads and help through the public links in this guide.